

Document number	P0403R1
Date	2016-11-09
Project	Programming Language C++, Library Evolution Working Group
Reply-to	Marshall Clow <mclow.lists@gmail.com>

Literal suffixes for `basic_string_view`

Introduction

When we adopted `basic_string_view` into the standard library from the Library Fundamentals TS, we did not define any literal suffixes for the type.

We adopted the `"s` suffix for `basic_string` to make it easy to declare `string` variables. We should do the same for `basic_string_view`. This would make declaring a `string_view` from a constant string simple:

```
std::string_view sv = "ABCD"sv;
```

These operators should be `constexpr`, since the constructors for `string_view` are `constexpr`.

Bikeshedding

In this paper, I am assuming that we will use the literal suffix `"sv` for this feature. Of course, this is subject to bikeshedding.

Wording

This is relative to N4606:

Add to the end of section [string.view.synop] (after "hash support")

```
inline namespace literals { inline namespace string_view_literals {  
  
// 21.4.X, _suffix for basic_string_view literals:_  
  
constexpr string_view    operator "" sv(const char* str, size_t len) noexcept;  
constexpr u16string_view operator "" sv(const char16_t* str, size_t len) noexcept;  
constexpr u32string_view operator "" sv(const char32_t* str, size_t len) noexcept;  
constexpr wstring_view   operator "" sv(const wchar_t* str, size_t len) noexcept;  
}}
```

Add a new section after [string.view.hash]:

[Note to the editor: The structure here is identical to [basic.string.literals]]

21.4.X Suffix for `basic_string_view` literals [basic.string_view.literals]

```
constexpr string_view operator "" sv(const char* str, size_t len) noexcept;  
  
a. Returns: string_view{str, len}.  
  
constexpr u16string_view operator "" sv(const char16_t* str, size_t len) noexcept;  
  
a. Returns: u16string_view{str, len}.  
  
constexpr u32string_view operator "" sv(const char32_t* str, size_t len) noexcept;  
  
a. Returns: u32string_view{str, len}.  
  
constexpr wstring_view operator "" sv(const wchar_t* str, size_t len) noexcept;  
  
a. Returns: wstring_view{str, len}.
```